

A3.1 SVM y Multiple Testing

En esta actividad trabajarás con la base de datos de la que se habló en clase, que consiste de 83 muestras y 2308 variables de entrada, que consisten en la expresión génica estandarizada de distintos genes. La variable de salida cuenta con valores numéricos del 1 al 4 que corresponden a distintos tipos de cáncer. Desarrolla los siguientes puntos en una Jupyter Notebook, tratando, dentro de lo posible, que cada punto se trabaje en una celda distinta. Los comentarios en el código siempre son bienvenidos, de preferencia, aprovecha el markdown para generar cuadros de descripción que ayuden al lector a comprender el trabajo realizado.

1. Importa los datos a tu ambiente de trabajo y revisa que no haya huecos. Calcula la diferencia de promedios entre las clases 2 y 4 para todos los genes, e imprime los 10 genes con la mayor diferencia de medias. Indica qué crees que esta diferencia podría implicar en términos de un estudio de inferencia.

Genes con grandes diferencias de media entre clases podrían ser marcadores potenciales de diferenciación biológica entre los tipos de cáncer.

```
In [4]: import pandas as pd
data = pd.read_csv('A3.1 Khan.csv')

# Revisión de huecos
missing = data.isnull().sum()
print("Filas totales:", len(data))
print("No hay valores faltantes")

gene_cols = [col for col in data.columns if col != 'y']

# Promedios por clase para las variables de expresión génica
grouped_means = data.groupby('y')[gene_cols].mean()

means2 = grouped_means.loc[2]
means4 = grouped_means.loc[4]

# Diferencia absoluta de medias entre clase 2 y 4
diff = (means2 - means4).abs()

# Tabla con promedios y diferencia, ordenar y mostrar top 10
top10 = pd.DataFrame({
    "mean_class2": means2,
    "mean_class4": means4,
    "abs_diff": diff
}).sort_values("abs_diff", ascending=False).head(10)

print("\nTop 10 genes con mayor diferencia absoluta de medias (clase 2 vs 4):")
display(top10)
```

Filas totales: 83
No hay valores faltantes

Top 10 genes con mayor diferencia absoluta de medias (clase 2 vs 4):

	mean_class2	mean_class4	abs_diff
X187	-1.487488	1.835663	3.323151
X509	-0.707556	2.198982	2.906537
X2046	-1.711089	0.713426	2.424515
X2050	-2.237496	0.164287	2.401783
X129	-1.759748	0.405437	2.165185
X1645	0.932217	-1.133242	2.065460
X1319	0.464020	-1.581922	2.045941
X1955	-0.912812	1.124528	2.037340
X1003	-1.807854	0.203483	2.011337
X246	1.161029	-0.676801	1.837830

2. Calcula el estadístico t y el p-value para comparar las medias de todos los genes entre la clase 2 y la clase 4 de la base de datos. Usa la metodología de Bonferroni, de Holm, y de Benjamini-Hochberg para corregir por múltiples pruebas e indica, para cada una, qué genes tienen una expresión significativamente distinta entre las clases (maneja un control de 0.05). Te recomiendo usar la función `multipletests` de `statsmodels.stats.multitest`

```
In [5]: from scipy.stats import ttest_ind
        from statsmodels.stats.multitest import multipletests
        import pandas as pd

        # Separar datos por clase
        g2 = data[data["y"] == 2][gene_cols].values
        g4 = data[data["y"] == 4][gene_cols].values

        # calcular estadístico t y p-values
        t_stats, p_vals = ttest_ind(g2, g4, axis=0, equal_var=False, nan_policy='propagate')

        # Crear DataFrame con resultados
        results = pd.DataFrame({
            'gene': gene_cols,
            't_stat': t_stats,
            'p_value': p_vals
        }).set_index('gene')

        # Metodos de corrección por pruebas múltiples
        methods = {
            'bonferroni': 'bonferroni',
```

```

        'holm': 'holm',
        'benjamini-hochberg': 'fdr_bh'
    }

    for name, method in methods.items():
        reject, pvals_corrected, _, _ = multipletests(results['p_value'].values, alpha=
        results[f'rechazar_{name}'] = reject
        results[f'pval_corr_{name}'] = pvals_corrected

    # Enseñar resumen de resultados
    for name in methods:
        sig = results.index[results[f'rechazar_{name}']].tolist()
        print(f"{name}: {len(sig)} genes significantes")
        if sig:
            print(sig)
        else:
            print("None")
    print("-" * 60)

```

bonferroni: 72 genes significant

['X2', 'X36', 'X67', 'X129', 'X174', 'X187', 'X188', 'X229', 'X246', 'X251', 'X338', 'X348', 'X368', 'X372', 'X373', 'X380', 'X430', 'X433', 'X509', 'X545', 'X554', 'X558', 'X566', 'X603', 'X655', 'X714', 'X762', 'X910', 'X951', 'X971', 'X1003', 'X1021', 'X1023', 'X1055', 'X1070', 'X1093', 'X1105', 'X1110', 'X1112', 'X1132', 'X1194', 'X1196', 'X1207', 'X1217', 'X1298', 'X1319', 'X1327', 'X1330', 'X1372', 'X1389', 'X1416', 'X1610', 'X1626', 'X1634', 'X1645', 'X1706', 'X1708', 'X1723', 'X1738', 'X1799', 'X1888', 'X1896', 'X1911', 'X1924', 'X1954', 'X1955', 'X1980', 'X2046', 'X2050', 'X2115', 'X2146', 'X2247']

holm: 72 genes significant

['X2', 'X36', 'X67', 'X129', 'X174', 'X187', 'X188', 'X229', 'X246', 'X251', 'X338', 'X348', 'X368', 'X372', 'X373', 'X380', 'X430', 'X433', 'X509', 'X545', 'X554', 'X558', 'X566', 'X603', 'X655', 'X714', 'X762', 'X910', 'X951', 'X971', 'X1003', 'X1021', 'X1023', 'X1055', 'X1070', 'X1093', 'X1105', 'X1110', 'X1112', 'X1132', 'X1194', 'X1196', 'X1207', 'X1217', 'X1298', 'X1319', 'X1327', 'X1330', 'X1372', 'X1389', 'X1416', 'X1610', 'X1626', 'X1634', 'X1645', 'X1706', 'X1708', 'X1723', 'X1738', 'X1799', 'X1888', 'X1896', 'X1911', 'X1924', 'X1954', 'X1955', 'X1980', 'X2046', 'X2050', 'X2115', 'X2146', 'X2247']

benjamini-hochberg: 296 genes significant

['X2', 'X3', 'X29', 'X36', 'X52', 'X67', 'X80', 'X89', 'X119', 'X129', 'X131', 'X139', 'X141', 'X146', 'X151', 'X153', 'X165', 'X166', 'X169', 'X174', 'X187', 'X188', 'X191', 'X214', 'X217', 'X229', 'X230', 'X244', 'X246', 'X251', 'X256', 'X258', 'X315', 'X323', 'X335', 'X336', 'X338', 'X340', 'X348', 'X365', 'X368', 'X372', 'X373', 'X378', 'X380', 'X384', 'X388', 'X390', 'X407', 'X409', 'X419', 'X426', 'X430', 'X433', 'X437', 'X443', 'X465', 'X469', 'X481', 'X482', 'X483', 'X500', 'X501', 'X503', 'X509', 'X518', 'X533', 'X534', 'X542', 'X545', 'X554', 'X558', 'X566', 'X598', 'X603', 'X604', 'X607', 'X636', 'X642', 'X650', 'X655', 'X696', 'X702', 'X714', 'X715', 'X733', 'X744', 'X746', 'X747', 'X753', 'X758', 'X760', 'X761', 'X762', 'X774', 'X779', 'X789', 'X795', 'X803', 'X808', 'X820', 'X828', 'X832', 'X851', 'X857', 'X867', 'X883', 'X891', 'X910', 'X941', 'X951', 'X964', 'X971', 'X973', 'X979', 'X982', 'X1003', 'X1012', 'X1020', 'X1021', 'X1022', 'X1023', 'X1030', 'X1037', 'X1039', 'X1046', 'X1055', 'X1063', 'X1070', 'X1074', 'X1089', 'X1090', 'X1093', 'X1100', 'X1105', 'X1110', 'X1112', 'X1113', 'X1125', 'X1132', 'X1151', 'X1155', 'X1158', 'X1160', 'X1164', 'X1187', 'X1194', 'X1196', 'X1201', 'X1203', 'X1204', 'X1206', 'X1207', 'X1217', 'X1220', 'X1225', 'X1227', 'X1228', 'X1238', 'X1252', 'X1254', 'X1258', 'X1263', 'X1283', 'X1286', 'X1292', 'X1298', 'X1301', 'X1319', 'X1325', 'X1327', 'X1330', 'X1331', 'X1345', 'X1353', 'X1372', 'X1389', 'X1416', 'X1464', 'X1489', 'X1493', 'X1496', 'X1497', 'X1498', 'X1517', 'X1518', 'X1525', 'X1536', 'X1574', 'X1597', 'X1606', 'X1610', 'X1613', 'X1626', 'X1634', 'X1643', 'X1644', 'X1645', 'X1646', 'X1655', 'X1670', 'X1671', 'X1673', 'X1677', 'X1691', 'X1697', 'X1706', 'X1708', 'X1714', 'X1723', 'X1727', 'X1729', 'X1734', 'X1738', 'X1771', 'X1772', 'X1775', 'X1778', 'X1799', 'X1821', 'X1822', 'X1831', 'X1850', 'X1853', 'X1854', 'X1855', 'X1857', 'X1867', 'X1870', 'X1878', 'X1882', 'X1888', 'X1896', 'X1901', 'X1906', 'X1909', 'X1910', 'X1911', 'X1914', 'X1917', 'X1920', 'X1924', 'X1929', 'X1931', 'X1937', 'X1942', 'X1945', 'X1954', 'X1955', 'X1967', 'X1979', 'X1980', 'X1991', 'X1994', 'X1995', 'X2000', 'X2020', 'X2039', 'X2046', 'X2047', 'X2049', 'X2050', 'X2060', 'X2080', 'X2081', 'X2083', 'X2086', 'X2088', 'X2093', 'X2096', 'X2099', 'X2105', 'X2114', 'X2115', 'X2117', 'X2120', 'X2146', 'X2148', 'X2149', 'X2159', 'X2172', 'X2181', 'X2199', 'X2227', 'X2230', 'X2235', 'X2247', 'X2248', 'X2253', 'X2262', 'X2275', 'X2278', 'X2295', 'X2300', 'X2301', 'X2303']

3. Realiza un experimento similar, pero ahora comparando las medias de las 4 clases de la base de datos. Para lograrlo, en vez de trabajar con el estadístico t , te recomiendo realizar pruebas de análisis de varianza (ANOVA). Dicha prueba la puedes realizar con la función `f_oneway` de `scipy.stats`, pero revisa bien cómo se deben ingresar los datos a dicha función, necesitarás primero estratificarlos por clase.

```
In [7]: from scipy.stats import f_oneway
import numpy as np
import pandas as pd

# ANOVA para comparar medias en las 4 clases (f_oneway)

# clases en orden consistente (usar la variable classes_set ya definida)
classes = sorted({1, 2, 3, 4})

n_genes = len(gene_cols)

F_stats = np.empty(n_genes)
p_vals_anova = np.empty(n_genes)

# calcular ANOVA por gen
for i, g in enumerate(gene_cols):
    # usar class_col (nombre de la columna de clase, p.ej. 'y') en lugar de la variable
    groups = [data.loc[data['y'] == k, g].values for k in classes]
    try:
        F, p = f_oneway(*groups)
    except Exception:
        F, p = np.nan, np.nan
    F_stats[i] = F
    p_vals_anova[i] = p

anova_results_4classes = pd.DataFrame({
    'gene': gene_cols,
    'F_stat': F_stats,
    'p_value': p_vals_anova
}).set_index('gene')

# Correcciones por pruebas múltiples (mismos métodos usados antes)
for name, method in methods.items():
    reject, pvals_corr, _, _ = multipletests(anova_results_4classes['p_value'].values,
                                             alpha=0.05, method=method)
    anova_results_4classes[f'reject_{name}'] = reject
    anova_results_4classes[f'pval_corr_{name}'] = pvals_corr

# Resumen de genes significativos por método
for name in methods:
    sig = anova_results_4classes.index[anova_results_4classes[f'reject_{name}']].tolist()
    print(f"{name}: {len(sig)} genes significantes")
    if sig:
        print("Primeros 50 genes significantes:", sig[:50])
        print("-" * 60)

# Mostrar top 10 genes por F-estadística
```

```
print("Top 10 genes por F-estadística:")  
display(anova_results_4classes.sort_values('F_stat', ascending=False).head(10))
```

bonferroni: 404 genes significantes

Primeros 50 genes significantes: ['X1', 'X2', 'X3', 'X17', 'X29', 'X33', 'X36', 'X50', 'X52', 'X54', 'X67', 'X74', 'X77', 'X84', 'X85', 'X94', 'X99', 'X107', 'X108', 'X119', 'X123', 'X127', 'X129', 'X139', 'X141', 'X142', 'X146', 'X151', 'X153', 'X165', 'X166', 'X169', 'X171', 'X174', 'X182', 'X187', 'X188', 'X192', 'X217', 'X229', 'X230', 'X235', 'X236', 'X239', 'X244', 'X246', 'X247', 'X248', 'X251', 'X255']

holm: 412 genes significantes

Primeros 50 genes significantes: ['X1', 'X2', 'X3', 'X17', 'X29', 'X33', 'X36', 'X50', 'X52', 'X54', 'X67', 'X74', 'X75', 'X77', 'X84', 'X85', 'X94', 'X99', 'X107', 'X108', 'X119', 'X123', 'X127', 'X129', 'X139', 'X141', 'X142', 'X146', 'X151', 'X153', 'X165', 'X166', 'X169', 'X171', 'X174', 'X182', 'X187', 'X188', 'X192', 'X217', 'X229', 'X230', 'X235', 'X236', 'X239', 'X244', 'X246', 'X247', 'X248', 'X251']

benjamini-hochberg: 1162 genes significantes

Primeros 50 genes significantes: ['X1', 'X2', 'X3', 'X9', 'X12', 'X17', 'X21', 'X22', 'X27', 'X29', 'X32', 'X33', 'X36', 'X37', 'X45', 'X46', 'X50', 'X52', 'X54', 'X56', 'X60', 'X64', 'X65', 'X66', 'X67', 'X68', 'X70', 'X73', 'X74', 'X75', 'X76', 'X77', 'X78', 'X80', 'X81', 'X84', 'X85', 'X88', 'X89', 'X90', 'X92', 'X93', 'X94', 'X95', 'X96', 'X97', 'X98', 'X99', 'X100', 'X107']

Top 10 genes por F-estadística:

	F_stat	p_value	reject_bonferroni	pval_corr_bonferroni	reject_holm	pval_corr_
gene						
X1955	84.364086	1.459035e-24	True	3.367454e-21	True	3.367454e-21
X1389	83.817537	1.772751e-24	True	4.091510e-21	True	4.089731e-21
X1003	77.795622	1.618988e-23	True	3.736625e-20	True	3.733381e-20
X2050	69.230799	4.733702e-22	True	1.092539e-18	True	1.091111e-18
X246	68.414042	6.633722e-22	True	1.531063e-18	True	1.528410e-18
X742	65.572797	2.195548e-21	True	5.067325e-18	True	5.056341e-18
X1	59.118264	3.839240e-20	True	8.860966e-17	True	8.837930e-17
X2162	56.987623	1.035143e-19	True	2.389109e-16	True	2.381861e-16
X1954	55.419914	2.182635e-19	True	5.037522e-16	True	5.020061e-16
X1645	54.768403	2.988392e-19	True	6.897208e-16	True	6.870311e-16

4. Separa los datos en entrenamiento y prueba, construye y entrena un modelo de SVM con un kernel lineal, con un kernel polinomial de orden 3, y con un kernel radial (puedes usar los parámetros que gustes, no necesitas optimizar con validación cruzada). Para evitar que el tiempo de procesamiento sea exagerado, puedes seleccionar solamente algunas variables, partiendo de los resultados que obtuviste en los puntos anteriores. Esta no es una práctica adecuada, pues estamos cayendo en una situación de fuga de datos. Lo ideal sería que la selección de características se basara solamente en experimentos realizados con los datos de entrenamiento. Pero, en este caso, obviaremos este detalle.

```
In [ ]: from sklearn.model_selection import train_test_split
        from sklearn.preprocessing import StandardScaler
        from sklearn.svm import SVC
        import time

        # Seleccionar top genes por F-stat
        top_n = 30
```

```

top_features = anova_results_4classes.sort_values('F_stat', ascending=False).head(t

# Preparar X, y
X = data[top_features].values
y = data['y'].values

# División entrenamiento/prueba (estratificada)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, stratify=y, random_state=42
)

# Escalado
scaler = StandardScaler()
X_train_s = scaler.fit_transform(X_train)
X_test_s = scaler.transform(X_test)

# Definir modelos SVM
models = {
    'svm_linear': SVC(kernel='linear', C=1.0, random_state=42),
    'svm_poly3': SVC(kernel='poly', degree=3, C=1.0, coef0=1.0, gamma='scale', rand
    'svm_rbf': SVC(kernel='rbf', C=1.0, gamma='scale', random_state=42)
}

# Entrenar y mostrar accuracy en entrenamiento y prueba
fit_times = {}
scores = {}

for name, clf in models.items():
    t0 = time.time()
    clf.fit(X_train_s, y_train)
    t1 = time.time()
    fit_times[name] = t1 - t0
    scores[name] = {
        'train_accuracy': clf.score(X_train_s, y_train),
        'test_accuracy': clf.score(X_test_s, y_test)
    }

# Resultados resumidos
print(f"Usando {top_n} features: {top_features}\n")
for name in models:
    print(f"{name}: fit_time={fit_times[name]:.3f}s, "
          f"train_acc={scores[name]['train_accuracy']:.3f}, "
          f"test_acc={scores[name]['test_accuracy']:.3f}")

```

Usando 30 features: ['X1955', 'X1389', 'X1003', 'X2050', 'X246', 'X742', 'X1', 'X216
2', 'X1954', 'X1645', 'X187', 'X545', 'X2022', 'X1194', 'X1319', 'X842', 'X107', 'X2
046', 'X153', 'X129', 'X554', 'X509', 'X1601', 'X1434', 'X1764', 'X566', 'X1427', 'X
1158', 'X1613', 'X2198']

svm_linear: fit_time=0.005s, train_acc=1.000, test_acc=1.000
 svm_poly3: fit_time=0.001s, train_acc=1.000, test_acc=1.000
 svm_rbf: fit_time=0.002s, train_acc=1.000, test_acc=1.000

5. Calcula, para los 3 modelos, las métricas que consideres importantes para comparar los desempeños. Indica qué opinas sobre los resultados, especificando si crees que uno de

los kernels es mejor para esta tarea específica.

En este caso, todos los kernels de SVM (lineal, polinomial de orden 3 y radial) lograron un desempeño perfecto en el conjunto de prueba, con una precisión del 100%. Esto sugiere que no hay un mejor kernel para esta tarea específica. Sin embargo, es importante considerar que este resultado podría deberse a la selección previa de características basada en los análisis estadísticos realizados, lo cual puede introducir un sesgo.

```
In [10]: from sklearn.metrics import classification_report, confusion_matrix, accuracy_score
for name, model in trained_models.items():
    print(f"\nResultados para {name}:")
    y_pred = model.predict(X_test_scaled)
    print("Matriz de confusión:")
    print(confusion_matrix(y_test_all, y_pred))
    print(f"Accuracy: {accuracy_score(y_test_all, y_pred):.3f}")
    print("\nReporte de clasificación:")
    print(classification_report(y_test_all, y_pred))
```

Resultados para svm_linear:

Matriz de confusión:

```
[[3 0 0 0]
 [0 7 0 0]
 [0 0 5 0]
 [0 0 0 6]]
```

Accuracy: 1.000

Reporte de clasificación:

	precision	recall	f1-score	support
1	1.00	1.00	1.00	3
2	1.00	1.00	1.00	7
3	1.00	1.00	1.00	5
4	1.00	1.00	1.00	6
accuracy			1.00	21
macro avg	1.00	1.00	1.00	21
weighted avg	1.00	1.00	1.00	21

Resultados para svm_poly3:

Matriz de confusión:

```
[[3 0 0 0]
 [0 7 0 0]
 [0 0 5 0]
 [0 0 0 6]]
```

Accuracy: 1.000

Reporte de clasificación:

	precision	recall	f1-score	support
1	1.00	1.00	1.00	3
2	1.00	1.00	1.00	7
3	1.00	1.00	1.00	5
4	1.00	1.00	1.00	6
accuracy			1.00	21
macro avg	1.00	1.00	1.00	21
weighted avg	1.00	1.00	1.00	21

Resultados para svm_rbf:

Matriz de confusión:

```
[[3 0 0 0]
 [0 7 0 0]
 [0 0 5 0]
 [0 0 0 6]]
```

Accuracy: 1.000

Reporte de clasificación:

	precision	recall	f1-score	support
1	1.00	1.00	1.00	3
2	1.00	1.00	1.00	7
3	1.00	1.00	1.00	5

4	1.00	1.00	1.00	6
accuracy			1.00	21
macro avg	1.00	1.00	1.00	21
weighted avg	1.00	1.00	1.00	21